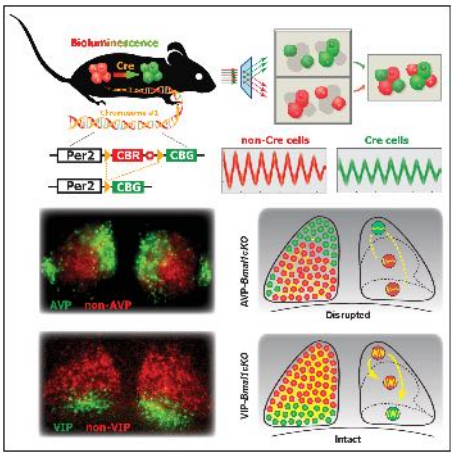
Coding Challenge 2 — Tuning a genetic oscillator — Modules: C/D

Background. Humans have long been perplexed by the passage of biological traits from one generation to the next. Mendel's experiments with peas laid the foundations of modern genetics, inspiring hopes that the relationship between genes and traits might be understood through a simple set of rules. More than a century later, we have discovered that these rules are only the beginning of a far deeper mystery.



Neural recordings from luminescent reporter.

Following decades of experiments in twentieth century molecular biology, we now recognize biological traits as the consequences of dynamic genetic processes. For instance, the time of day when a gene reaches its peak expression may be more consequential to the organism than the gene's average expression level. Unfortunately, it can be very challenging to measure gene expression dynamics without inadvertently perturbing the very system we wish to study.

Direct measurements, such as blood or tissue sampling, can be invasive and difficult to repeat frequently. An alternative is to attach a **luminescent reporter** to a protein of interest, allowing its abundance to be monitored continuously using highly sensitive light detectors. However, these modifications can induce unintended changes in gene expression. In this challenge, we will use ODE models to investigate how genetic engineering can help minimize these unwanted side-effects of luminescent reporters.

Mathematical model. The ODE systems we study in this coding challenge are known as **gene regulatory network** (GRN) models:

- Each variable represents the concentration of a molecular species, such as a protein or messenger RNA (mRNA), averaged across a family of cells.
- The parameters describe the rates of the underlying biochemical processes.

The genetic processes in GRN models arise from the **central dogma of molecular biology**, which describes how genetic information stored in DNA is used to produce the proteins necessary for life:

- A segment of DNA that encodes a particular protein is called a **gene**.
- During **transcription**, an enzyme called RNA polymerase uses the information in a gene to produce messenger RNA (mRNA).
- During **translation**, the information in mRNA is used to synthesize the corresponding protein.
- Proteins can create **feedback loops** when they interact with DNA and other regulatory machinery.

Most genes undergo **regulated gene expression**, meaning that their expression is influenced by other molecular factors within the cell. These interactions are commonly classified into two types:

- **Repression:** Increasing the concentration of a regulatory factor decreases the expression of the gene.
- **Activation:** Increasing the concentration of a regulatory factor increases the expression of the gene.

Setup. The technical focus of this challenge will be on **parametric sensitivity analysis**, which studies how changes in model parameters affect the behaviour of ODE systems. As you work through this challenge, you will become familiar with:

- How living cells use non-linear feedback to generate *autonomous oscillations*.
- The influence of a concept called *numerical stiffness* on the performance of different numerical methods.
- How parametric sensitivity analysis can guide the design of wet-lab experiments.

Autonomous oscillations are what makes the concentration of a given protein have a cyclical behaviour (like "increase from the early morning until late afternoon, then decrease until early morning, repeat").

Q1: From genetic feedback to autonomous oscillations (1+1+2.5 public points)

How can a family of proteins generate oscillations in the first place? In the 1960s, this question motivated the development of the **Goodwin model**. This model was based around the intuition that oscillations should be possible from a gene with sufficiently nonlinear regulation and a long enough effective time-delay.

We consider the generalized form of the Goodwin model which consists of chemical species $x_1, \ldots, x_N$ of which x_1 is the main focus.

$$x_1' = \frac{\alpha}{1 + x_N^H} - x_1, \qquad H \geq 0 \qquad\qquad x_i' = x_{i-1} - x_i, \qquad i = 2, \ldots, N.$$

Notice that our model has the structure of a feedback loop since each x_i only interacts with x_{i-1} and x_{i+1}. Each of the candidate features for generating oscillations is represented by a distinct parameter in the model:

- **nonlinear regulation** – the model is nonlinear if and only if $H > 0$.
- **time-delay** – the length of our feedback loop is controlled by N, and this serves as an effective time-delay in the model (how long it takes for a spike in x_1 to propagate through the whole loop).

We will use numerical simulations to explore the simplest possible genesis of biochemical oscillations. While your functions should work for any value of α, we will use $\alpha = 10$ throughout the simulation.

(a) **Implement the model.** Implement `goodwin_rhs(t, x, alpha, H)` so that it returns the derivative vector `dx = [dx1, ..., dxN]`. Your function should infer N from the length of `x`, so the function works for arbitrary loop lengths.

The `plot_exercises` function (already implemented in `q1.py`) simulates the model for $N = 3$, $H = 2$ and two different initial states. It also produces a plot. Compare the trajectories and late-time states. Do they approach the same equilibrium, as expected for this model?

(b) **Investigate effective time-delay.** By simulating the model, we will develop a better understanding of how N represents an effective time-delay. Implement a function `feedback_delay(N, alpha, H, threshold, end)` that uses initial condition $x_1(0) = 1$ and $x_i(0) = 0$ for $i = 2, \ldots, N$ and then determines the first time x_N exceeds the `threshold`, or returns `NaN` if it is not exceeded by the time `end`.

The `plot_exercises` function uses threshold $1/10$, $H = 8$, and a total simulation time of 300 and evaluates your function for several values of N. If we imagine that x_1 is waiting to feel the "pulse" cycle through the full loop, we see that this waiting time increases with N, the literal length of the loop.

(c) **Detect oscillation.** Depending on the values of N and H, the solution of the system either settles to a *steady state* or keeps oscillating forever (a *sustained oscillation*). Your task is to implement a function `oscillation_stats(N, H, alpha, discard, end, samples)` that tells these two cases apart automatically, and then use it to map out which parameter values give oscillations.

Solve the system numerically from $t = 0$ up to a time given by `end`. Track the times at which x_1 reaches successive peaks and discard the first few peaks (number given by `discard`) to reduce transient effects. Retain at least four peaks after discarding the transient peaks. Let Δt_k ($k = 1, \ldots, n$) denote the remaining peak-to-peak intervals. Compute their mean $\widehat{\Delta t}$ (i.e. the estimated period) and their sample standard deviation $s_{\Delta t}$.

Classify the motion as a sustained oscillation when the *coefficient of variation* $s_{\Delta t}/\widehat{\Delta t}$ is at most 0.01, the *peak-to-trough amplitude* A_n over the last cycle is at least 10^{-2}, and the *relative change* of this quantity from the preceding cycle $|A_n - A_{n-1}|/\max(A_n, A_{n-1})$ is at most 0.1%. In this case, the function should return $(\widehat{\Delta t}, s_{\Delta t})$. If any of these conditions are not fulfilled, the function should report no sustained oscillations by returning (NaN, NaN).

The `plot_exercises` function produces a phase plot showing where sustained oscillations occur in the (N, H) parameter space.

Q2: When biological clocks become stiff (2 public + 1 hidden points)

Many ODE systems have **multiscale dynamics**, meaning that processes evolve on widely different timescales. This is particularly common in gene regulatory networks, where mRNA transcription and degradation can occur over minutes to hours, while protein synthesis and degradation may take hours to days.

The presence of these different timescales can have important consequences for numerical simulation. The methods you've encountered in class are all examples of **explicit time-stepping**, but an alternative approach, known as **implicit time-stepping**, can be more suitable for multiscale systems. The distinction comes down to the following trade-off:

- **Explicit methods:** Each time-step is relatively cheap to compute, but numerical stability may require very small time-steps when the ODE has multiple timescales.
- **Implicit methods:** Each time-step is more expensive to compute, but numerical stability can often be maintained with much larger time-steps even when the system is multiscale.

As a result, implicit methods can be considerably more efficient despite their higher cost per step. This trade-off is closely related to the concept of **numerical stiffness**. Although a precise definition is beyond the scope of this course, we can think of a stiff system as one where numerical stability forces explicit methods to take much smaller time-steps than would otherwise be needed for accuracy.

We will investigate numerical stiffness using the Goldbeter model of the *Drosophila* (fruit fly) circadian clock. Unlike the abstract Goodwin model, the Goldbeter model describes a specific negative feedback loop involving the clock gene *per*. This gene produces a protein in the cytoplasm that undergoes a sequence of chemical reactions before entering the nucleus, where it inhibits further production of its own gene. By doing so, the Goldbeter model captures the same basic feedback mechanism as the Goodwin model, but in greater biological detail.

To investigate stiffness, we will vary the speed of these intermediate reactions and compare two numerical solvers:

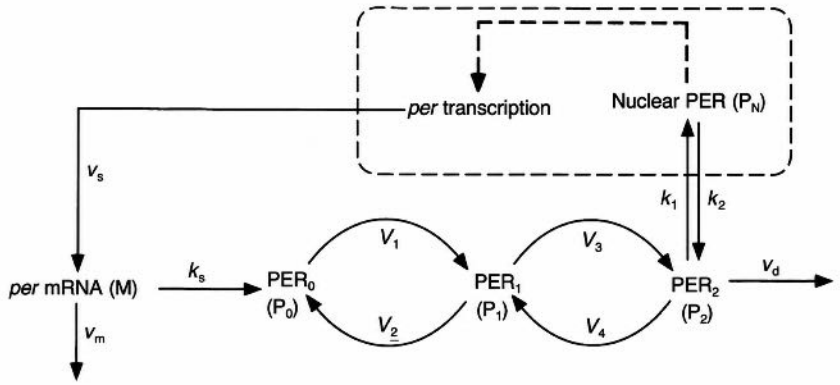
- `RK45`: An **explicit Runge–Kutta method** that computes each step using previously calculated values.
- `Radau`: An **implicit Runge–Kutta method** that solves a system of equations at each step, allowing it to handle stiffness more effectively.

In the following exercises, you will add timescale separation to the supplied model and investigate how changing the reaction timescales affects the performance of these two methods.

(a) **Introducing multiple timescales.** Implement the unscaled model in `goldbeter_rhs` using the full-model equations below. Implement `goldbeter_rhs_epsilon(t, x, p, epsilon)` as a wrapper: make a copy of the parameter dictionary, scale each of $v_1, \ldots, v_4$ by $1/\epsilon$, and pass the copy to `goldbeter_rhs`. The supplied `simulate(epsilon, method)` numerically solves the system for the given method, and `estimate_period(solution)` estimates the oscillation period from late peaks of nuclear PER, P_N. The supplied `plot_exercises` uses `RK45` for 100 hours with $\epsilon \in \{1, 10^{-1}, 10^{-2}, 10^{-3}\}$ and the tolerances in `q2.py`. How does the period change as the intermediate reactions become faster?

(b) **Diagnosing stiffness.** Implement `rk45_step_count(epsilon)` and `radau_step_count(epsilon)` to return the number of time steps used by each solver on the full model. The supplied `plot_exercises` compares both counts against ϵ on logarithmic axes. How does the RK45 step count change as the reactions become faster?

(c) **Optional rapid-equilibrium illustration (no implementation required).** The supplied code also includes a reduced model in which the fast phosphorylation subsystem is assumed to be at equilibrium. The plotting routine compares its error and solver step count with the full model as ϵ decreases. You do not need to modify `fast_equilibrium`, `reduced_rhs`, `reduced_error`, or `reduced_step_count`; this part is provided for context and visual inspection only.



Goldbeter's PER clock network, showing repression by nuclear PER, sequential phosphorylation, degradation, and nuclear transport.

The full model has state (M, P_0, P_1, P_2, P_N), with phosphorylation fluxes

$$J_1 = \frac{v_1 P_0}{\epsilon(k_1 + P_0)}, \qquad J_2 = \frac{v_2 P_1}{\epsilon(k_2 + P_1)},$$
$$J_3 = \frac{v_3 P_1}{\epsilon(k_3 + P_1)}, \qquad J_4 = \frac{v_4 P_2}{\epsilon(k_4 + P_2)}.$$

Its equations are

$$\begin{aligned}
\dot{M} &= \frac{v_s}{1 + (P_N/k_i)^n} - \frac{v_m M}{k_{m1} + M}, \\
\dot{P}_0 &= k_s M - J_1 + J_2, \\
\dot{P}_1 &= J_1 - J_2 - J_3 + J_4, \\
\dot{P}_2 &= J_3 - J_4 - k_{t1} P_2 + k_{t2} P_N - \frac{v_d P_2}{k_d + P_2}, \\
\dot{P}_N &= k_{t1} P_2 - k_{t2} P_N.
\end{aligned}$$

Q3: Parametric sensitivity of an oscillating system (0.5+0.5+0.5 public points)

We now turn our attention to **sensitivity analysis**. Given some quantity F that depends on a model parameter p, we want to understand how much F changes when p is perturbed. One way to quantify this dependence is through the dimensionless sensitivity coefficient

$$S_{F,p} = \frac{p}{F}\frac{\partial F}{\partial p}.$$

This measures the fractional change in F resulting from a small fractional change in p.

For an oscillating system, one particularly important quantity is the **period** T. We will investigate how the period of the Goldbeter model depends on the parameter v_d, which controls the rate of PER degradation. Since the period is obtained numerically, we cannot simply differentiate an explicit formula for $T(v_d)$. Instead, we will use **finite differences** to approximate its derivative.

Recall that the centered finite-difference approximation is

$$D_2(h) = \frac{T(v_d + h) - T(v_d - h)}{2h}.$$

For sufficiently small h, this approximation has an error of order $O(h^2)$. We might therefore expect that making h smaller always improves the approximation. However, numerical errors eventually affect the estimated periods significantly, limiting the accuracy we can achieve.

A useful technique for improving finite differences is **Richardson extrapolation**. It can be shown that by combining approximations computed with two different step sizes, certain parts of the error cancel. This is why

$$D_4(h) = \frac{4D_2(h/2) - D_2(h)}{3}.$$

is a fourth-order approximation, with error $O(h^4)$.

(a) **Estimating the period.** Implement `estimate_period(vd)` using the initial condition and parameter values in `q2.py`, replacing the nominal v_d with the supplied argument. Use the supplied `simulate(vd)` to integrate for 120 hours with its provided settings, and `_period_from_solution(sol)` to measure the period.

The `_period_from_solution(sol)` function estimates T as the mean of the final three peak-to-peak intervals of nuclear PER, p_N, provided their coefficient of variation is at most 0.01. Otherwise it returns `NaN`. The `plot_exercises` plots the solutions and evaluates the period at the nominal parameter value and at $v_d \pm 0.01$. How does increasing v_d affect the oscillation period?

(b) **Finite-difference sensitivity.** Implement `finite_difference(h)` to approximate $\partial T/\partial v_d$ using the centred finite-difference formula $D_2(h)$ and your `estimate_period` function of part (a).

The `plot_exercises` function evaluates your approximation for $h \in \{0.5, 0.1, 0.05, 0.01, 0.005, 0.001\}$, and computes the corresponding dimensionless sensitivity coefficients $S_{T,v_d} = (v_d/T)D_2(h)$. How does the estimated sensitivity change as h decreases? Why might choosing an extremely small step size be problematic?

(c) **Improving the approximation.** Implement `richardson(h)` to estimate $\partial T/\partial v_d$ with the fourth-order approximation $D_4(h)$ of the Richardson extrapolation and your `estimate_period` function of part (a).

The `plot_exercises` function compares the Richardson and centred estimates of the dimensionless sensitivity coefficient for the same step sizes as part (b). Compare the results of part (b) and part (c). Which method reaches a stable estimate at a larger step size? Why might neither method continue to improve for arbitrarily small h?

Challenge (3 public points)

The challenge will be published within the next few days.